Maîtriser les fonctionnalités avancées de React

Réutilisation du code à l'aide de la composition

Définition

En React, **la composition** consiste à assembler plusieurs composants pour créer un composant plus complexe.
C'est une manière élégante de **réutiliser le code**, au lieu d'utiliser l'héritage comme dans d'autres langages (Java, C++, etc.).

L'idée :
Un composant peut **recevoir d'autres composants** via la **prop spéciale children**.

Exemple 1 – Composant de base réutilisable : Card

Code :

```
import React from "react";

// Un composant Card réutilisable

function Card({ title, children }) {

  return (

    <div
```

```
      style={{
        border: "1px solid #ccc",
        padding: "10px",
        borderRadius: "8px",
        marginBottom: "10px",
        boxShadow: "0px 2px 5px rgba(0,0,0,0.1)",
      }}
    >
      {/* Si un titre est fourni, on l'affiche */}
      {title && <h3>{title}</h3>}

      {/* children représente le contenu interne du composant
*/}
      <div>{children}</div>
    </div>
  );
}

export default Card;
```

Explication :

- props.children : mot clé spécial en React, qui contient **tout le contenu placé entre les balises** du composant.
- Le composant Card n'a pas besoin de connaître à l'avance ce qu'il va contenir → il est **générique** et **réutilisable**.

Utilisation :

```
import React from "react";
import Card from "./Card";

function App() {
  return (
    <div>
      <Card title="Profil utilisateur">
        <p>Nom : Hafid</p>
        <p>Rôle : Formateur</p>
      </Card>

      <Card title="Statistiques">
        <ul>
          <li>Projets : 3</li>
          <li>Tâches : 7</li>
```

```
      </ul>
     </Card>
    </div>
  );
}

export default App;
```

Exemple 2 – Composition imbriquée

```
function Container({ children }) {
  return <div style={{ padding: "20px", backgroundColor: "#f5f5f5" }}>{children}</div>;
}

function Page() {
  return (
    <Container>
     <Card title="Bienvenue">
      <p>Contenu principal de la page.</p>
     </Card>
```

```
    </Container>
  );
}
```

Ici, Page compose deux composants : Container et Card.

Routage dans React

Définition : qu'est-ce que le routage ?

Le **routage** permet de naviguer entre plusieurs pages (ou vues) sans recharger toute l'application.
React utilise le concept de **SPA (Single Page Application)**.

Étape 1 – Installation de React Router

npm install react-router-dom

Étape 2 – Configuration de base

Structure :

```
Src/
 — App.jsx
 — index.js
 — pages/
```

— Home.jsx

— About.jsx

— Contact.jsx

Code index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";

const root =
ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

BrowserRouter : composant parent qui active le système de routage.

Code App.jsx

```
import React from "react";
import { Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

export default function App() {
  return (
    <div>
      {/* Barre de navigation */}
      <nav>
        <Link to="/">Accueil</Link> |{" "}
        <Link to="/about">À propos</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      {/* Zone principale où les pages s'affichent */}
      <Routes>
        {/* Chaque Route correspond à une page */}
        <Route path="/" element={<Home />} />
```

```
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </div>
 );
}
```

Explications :

- <Link> remplace <a href> → pas de rechargement de la page.
- <Routes> contient plusieurs <Route> qui associent **un chemin** (path) à **un composant** (element).
- element={<Home />} indique le composant à afficher.

Routage imbriqué (Nested Routes)

Certaines pages peuvent avoir leurs **propres sous-pages**.

Exemple :

/produits

/produits/:id

```
import { Routes, Route, Outlet, Link, useParams } from "react-router-dom";
```

```
function Produits() {
  return (
    <div>
      <h2>Liste des produits</h2>
      <ul>
        <li><Link to="1">Produit 1</Link></li>
        <li><Link to="2">Produit 2</Link></li>
      </ul>
      {/* Outlet = emplacement où la sous-route sera affichée */}
      <Outlet />
    </div>
  );
}

function ProduitDetails() {
  const { id } = useParams(); // Récupère l'ID depuis l'URL
  return <h3>Détails du produit {id}</h3>;
}

export default function App() {
```

```
  return (
    <Routes>
      <Route path="/produits" element={<Produits />}>
        {/* Sous-routes */}
        <Route path=":id" element={<ProduitDetails />} />
      </Route>
    </Routes>
  );
}
```

Outlet = zone d'affichage du contenu des sous-routes.
useParams() = hook pour récupérer les paramètres dynamiques de l'URL.

Utilisation des Hooks

Définition :

Les **Hooks** sont des fonctions spéciales permettant d'ajouter des fonctionnalités React (état, cycle de vie, etc.) dans les composants fonctionnels.
Introduits depuis **React 16.8**.

Voir le lien :
https://www.w3schools.com/react/react_hooks.asp

useState() – gérer un état local

```
import React, { useState } from "react";
function Compteur() {
  const [count, setCount] = useState(0); // Déclare une variable d'état

  return (
    <div>
      <p>Valeur actuelle : {count}</p>
      <button onClick={() => setCount(count + 1)}>Incrémenter</button>
    </div>
  );
}

export default Compteur;
```

useState retourne un tableau :

1. La valeur actuelle (count)
2. Une fonction pour la modifier (setCount)

useEffect() – effets secondaires

Permet d'exécuter du code après le rendu du composant.

```
import React, { useState, useEffect } from "react";

function Horloge() {
  const [time, setTime] = useState(new
Date().toLocaleTimeString());

  useEffect(() => {
    const timer = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    // Nettoyage à la fin
    return () => clearInterval(timer);
  }, []);

  return <h2>Heure : {time}</h2>;
}
```

```
export default Horloge;
```

useEffect :

- 1er paramètre : fonction à exécuter.
- 2e paramètre : liste des dépendances (ici vide → exécuté une seule fois).

useRef() – garder une référence sans re-render

```
import React, { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus(); // Accès direct à l'élément DOM
  };

  return (
    <div>
```

```
      <input ref={inputRef} type="text" placeholder="Tapez ici"
/>
      <button onClick={handleFocus}>Focus</button>
    </div>
  );
}
```

useContext() – partager un état global

```
import React, { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function Bouton() {
  const theme = useContext(ThemeContext);
  return <button>{theme === "light" ? "Clair" :
"Sombre"}</button>;
}

function App() {
  return (
```

```
    <ThemeContext.Provider value="dark">
      <Bouton />
    </ThemeContext.Provider>
  );
}
```

Tests sur les composants

Définition :

Les tests permettent de **vérifier automatiquement** que les composants fonctionnent comme prévu.
React utilise souvent :

- **Jest** (moteur de test)
- **React Testing Library** (simulation du rendu React)

Exemple de test simple

Fichier Compteur.jsx

```
import React, { useState } from "react";

export default function Compteur() {
  const [count, setCount] = useState(0);
```

```
  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Fichier Compteur.test.jsx

```
import { render, screen, fireEvent } from "@testing-library/react";
import Compteur from "./Compteur";


test("le compteur s'incrémente quand on clique sur le bouton", () => {
  render(<Compteur />);
  const button = screen.getByText("+");
  fireEvent.click(button);
  expect(screen.getByText("Compteur : 1")).toBeInTheDocument();
});
```

Explication :

- render() : affiche le composant en mémoire.
- screen.getByText() : recherche un texte à l'écran.
- fireEvent.click() : simule un clic.
- expect(...).toBeInTheDocument() : vérifie que le résultat attendu existe.